

Two routes to interpretability

You built models that work — but *why* do they predict what they do?

Two routes:

- ▶ **Use a glass-box model you can read** — the model *is* the explanation. **(this deck)**
- ▶ **Explain a black box afterwards** with model-agnostic tools. (decks 2–3)

Today: the two transparent families you already know — **linear models** and **trees**.

Why bother? To **trust** a model, **debug** it, and catch ones that *cheat* — like deck 3's “wolf” detector that really keys on *snow*.

glass box: *read it*

linear, tree, rules

black box: *explain it*

forest, boosting, NN

Predict-first: which one can you actually read?

A linear model and a random forest reach the **same accuracy** on the bike-rental data.
Which can you *read* — and how?

Predict-first: which one can you actually read?

A linear model and a random forest reach the **same accuracy** on the bike-rental data. Which can you *read* — and how?

- ▶ **Linear model:** yes — each feature has one coefficient.
- ▶ **A single tree:** yes — follow the splits from root to leaf (a rule).
- ▶ **A random forest:** **no** — hundreds of trees, no readable whole. But it still hands you a built-in *importance* number (which, we'll see, is biased).

Running example: **bike sharing** — predict daily rental count from weather & calendar features.

Outline

Reading a linear model

Reading trees

Rule-based models

So which features matter?

Reading a linear model

The most transparent model: one number per feature.

Coefficients as importance

A linear model $\hat{y} = \theta_0 + \sum_j \theta_j x_j$ tells you everything directly:

- ▶ **Sign** of θ_j = direction; **magnitude** = effect per +1 unit of x_j .
- ▶ **Standardize first** to compare — a raw coef of 5000 on temp (range 0–1) vs 2 on hum is *not* “2500× more important”; that’s units (L12). Standardized, $|\theta_j|$ is fair: on bike yr & atemp top it.
- ▶ Classification analog: e^{θ_j} = **odds ratio** (L11).

No extra method needed: a (standardized) coefficient *is* a built-in importance and effect, in one number.

Linear models: the caveats

Only additive, linear effects

- ▶ One straight-line effect per feature; no interactions or curves unless you engineer them.

Correlated features split the credit

- ▶ temp and atemp (feels-like) are nearly the same; the model may give each half the coefficient — or unstable, even opposite, signs.

A coefficient answers “how does the prediction move with this feature, *holding the others fixed*” — which is exactly what breaks when features move *together*.

Lasso: the model selects its own features

L1 (Lasso) regularization shrinks coefficients toward zero — and drives many to **exactly zero** (callback: Ch. 2 regularization).

- ▶ Non-zero coefficients = the features the model *kept* — a built-in, sparsity-based **feature selection**.
- ▶ Stronger penalty $\Rightarrow$ fewer survivors. **On bike (scaled)**: the CV-tuned Lasso keeps 10/11; turn the penalty up and survivors drop $10 \rightarrow 4 \rightarrow 1$ — **sparsity is a knob you set**.

Caveat: among correlated features, Lasso keeps *one* and zeros the rest *arbitrarily* — “not selected” does not mean “unimportant.”

Reading trees

One tree you can read; a forest you cannot — but it still scores features.

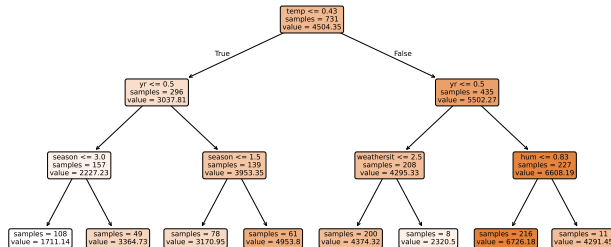
A single tree is a glass box

Each **root-to-leaf path** is a rule:

*warm day ($temp > 0.43$), year 2,
good weather $\rightarrow$ predict ~ 6726
rentals.*

- Fully transparent — if it stays shallow.
- Depth $\uparrow \Rightarrow$ accuracy $\uparrow$ but readability $\downarrow$.

A shallow regression tree on bike rentals (readable)



How a split is chosen: reduce impurity

A tree picks the split that most reduces **impurity**: $\text{reduction} = \text{parent impurity} - \text{weighted child impurity}$. Two impurity measures:

Classification: information gain (entropy H)

Toy: parent 5/5 $\Rightarrow H = 1.0$. A 4/1 child:
 $-\frac{4}{5} \log_2 \frac{4}{5} - \frac{1}{5} \log_2 \frac{1}{5} = 0.72$.

$H_{\text{child}} = \frac{1}{2}(0.72) + \frac{1}{2}(0.72) = 0.72$, $\text{gain} = 1.0 - 0.72 = \mathbf{0.28}$

Regression: variance reduction (bike root split)

Split temp ≤ 0.43 ($n = 731$):

$$\text{Var}_{\text{parent}} \approx 3.75\text{M}$$

$$\text{Var}_{\text{child}} = 0.40(2.24\text{M}) + 0.60(2.31\text{M}) = 2.28\text{M}$$

$$\text{reduction} \approx \mathbf{1.46\text{M}} \ (\approx 39\%)$$

Same idea, two rulers — the split with the biggest reduction wins.

Forests: unreadable, but they score features

A random forest is **hundreds of trees** — no readable whole. Yet it gives a built-in **feature importance**:

Sum each feature's impurity reduction (info gain / variance reduction) over *all* its splits, averaged across the trees $\Rightarrow$ one importance per feature.

But it is biased:

- ▶ Favors **high-cardinality** / **continuous** features (more places to split).
- ▶ **Dilutes** across correlated features (credit shared between temp, atemp).
- ▶ Computed on **training** data $\Rightarrow$ can reward overfitting.

Reading these in sklearn

```
# linear: standardized coefficients as importance
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, LassoCV
from sklearn.pipeline import make_pipeline
Xs = StandardScaler().fit_transform(X)
coef = LinearRegression().fit(Xs, y).coef_           # |coef| =
           importance
sel = make_pipeline(StandardScaler(), LassoCV())     # SCALE, then
           L1 (Lasso is scale-sensitive)
kept = sel.fit(X, y)[-1].coef_ != 0                 # features
           Lasso keeps (alpha tuned by CV)

# trees: built-in (impurity) importance + read one tree
from sklearn.ensemble import RandomForestRegressor
from sklearn.tree import DecisionTreeRegressor, plot_tree
RandomForestRegressor().fit(X, y).feature_importances_
plot_tree(DecisionTreeRegressor(max_depth=3).fit(X, y))
```

Rule-based models

Rules you can read — a tree's logic with a linear model's sparsity.

RuleFit: turn rules into features

A single tree is readable but brittle; a forest is accurate but unreadable. **RuleFit** goes for both, in three steps:

1. **Mine rules** from a tree ensemble: every root-to-node path becomes a binary rule — e.g. $(\text{temp} > 0.35 \text{ and } \text{yr} = 1)$, which is 1 when the row falls in that region and 0 otherwise.
2. **Fit a sparse linear model** (Lasso) over *both* the original features and all these new rule-features.
3. **Keep the non-zero terms** — a short, weighted list of rules (plus a few surviving linear terms).

RuleFit = **rules from trees** + **Lasso selection** — the two glass boxes from this deck, combined into one readable model.

Reading a RuleFit model: a scorecard

On bike (baseline $\approx$ the average, ~ 4500 rentals), read it like a **scorecard**: start at the baseline, then add the weight of every rule that *fires* for this day.

rule (if it fires, add the weight)	weight
temp > 0.35 and yr = 1 and hum $\leq$ 0.88	+551
atemp > 0.29 and yr = 1	+266
temp $\leq$ 0.43	-256

A warm 2012 day fires the first two (+817); a cold day fires the third (-256). This 13-rule model scores $R^2 \approx 0.65$.

Watch out:

- ▶ More rules $\Rightarrow$ more accurate but less readable — the tree's depth/readability dial again (a denser RuleFit climbs toward the forest's ~ 0.87).
- ▶ Overlapping rules **share credit**: one weight is not a feature's full effect.

Predict-first: do the two methods agree?

Same bike data, same goal. **Predict:**
will the linear model and the random
forest pick the *same* top features?

Predict-first: do the two methods agree?

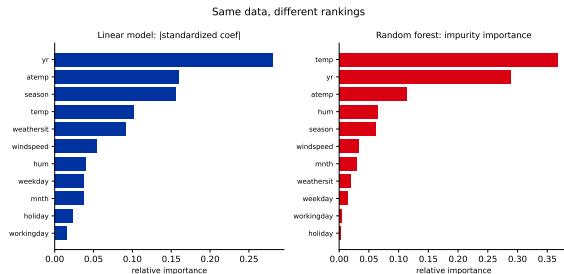
Same bike data, same goal. **Predict:** will the linear model and the random forest pick the *same* top features?

They don't:

- ▶ Linear: yr, atemp, season on top.
- ▶ Forest: temp, yr, atemp on top.

Part of the gap is the **impurity bias** we just saw; the rest is that they genuinely **measure different things** (a linear effect vs. impurity reduction), splitting the temp/atemp correlation differently.

Importance is method-dependent — always say *which* importance you mean.



Recap

- ▶ **Linear models** are read directly: (standardized) **coefficients** = effect + importance; e^{θ_j} = odds ratio; **Lasso** selects features by zeroing them.
- ▶ **A single tree** is a glass box (paths = rules); splits are chosen by **impurity reduction** (information gain / variance reduction).
- ▶ **RuleFit** mines rules from an ensemble, then Lasso-selects a short weighted list you read as a **scorecard** — accuracy vs readability is a dial you set.
- ▶ **Forests** are black boxes but emit a built-in **impurity importance** — fast, but **biased** (cardinality, correlation, train-based).
- ▶ Different methods give **different rankings** — importance is method-dependent, and *none of it is causation*.

Next: model-agnostic interpretability — **permutation importance** (fixes the cardinality & train-based biases — though correlation still bites) and **feature effects** (PDP / ICE), for *any* model.